

InsightFlow Agent 开发计划文档

1. 文档目标

本文档用于指导 InsightFlow Agent 的实际开发。

InsightFlow Agent 的最终定位是：

基于 LangGraph 的 Multi-Agent Tool-Calling BI Workflow

项目第一目标不是做普通 Text2SQL，也不是做普通数据分析助手，而是突出：

Multi-Agent 协作
Tool Calling
SQL Execution
Execution Feedback Repair
Trace
Eval

开发原则：

P0 先做小而稳的 Agentic SQL Core
P1 再做可靠分析和报告生成
P2 再做经营周报与行动闭环
P3 最后做 MCP、API、异步和工程化

2. 参考项目策略

开发时只重点参考 2 个主项目，避免项目杂交过多。

2.1 P0 主参考：adamfaik/sql-agent

用于参考：

LangGraph SQL workflow
SQLite SQL 执行
SQL 执行失败后的 self-correction
Streamlit glass-box demo
Eval benchmark
电商数据分析问题设计

InsightFlow 需要吸收它的：

SQL 执行工具设计
数据库错误捕获
失败后重试逻辑
Streamlit 展示 SQL / error / retry / result
Eval 测试集组织方式

但不要照搬它的：

单文件式 agent 结构
偏 Text2SQL 的项目包装
缺少 Agent / Tool 分层的地方

InsightFlow 应该把它拆成：

agents/
tools/
graph/
eval/
reports/
tests/

2.2 P1 / P3 主参考：mallahyari/langgraph-sql-agent

用于参考：

Router / Table Selector / SQL Generator / Validator / Executor / Visualization Planner
LangGraph 条件边
图表生成 workflow
backend / frontend 分层
FastAPI + React + SSE 展示
工程化目录结构

InsightFlow 需要吸收它的：

多 Agent 模块化拆分
Visualization Planner 思路
Graph 条件边设计
前端展示 agent steps 的方式
P3 的 FastAPI / SSE 思路

但不要照搬它的：

较弱的 SQL Validator
一开始就做 React + FastAPI + SSE
过重的前后端架构

InsightFlow P0 仍然使用 Streamlit。

2.3 局部参考：azain47/Multi-Agent-Text2SQL-System

只参考：

sqlglot / parser-based SQL validation
Feedback Formatter
iterative feedback loop
max retry / feedback history

不作为主参考，因为它更偏 SQL 生成与验证，不是完整的真实数据库执行闭环。

InsightFlow 中可以借鉴它增强：

validate_sql()
Error Fix Agent 的错误反馈格式
SQL Reviewer 的字段 / 表名检查

3. 技术栈

3.1 P0 技术栈

Python 3.11+
LangGraph
LangChain Core
SQLite
SQLAlchemy 或 sqlite3
Pydantic
Streamlit
pytest
python-dotenv
sqlparse
sqlglot
pandas

P0 暂不做：

MCP
FastAPI
React
异步任务队列
RBAC
多数据源
复杂 Dashboard

3.2 P1 技术栈

matplotlib 或 plotly
Markdown report
pandas
简单 keyword-based context retrieval
YAML / JSON / Markdown context files

3.3 P2 技术栈

SQLite action tables
approval records
audit logs
task / alert persistence

3.4 P3 技术栈

MCP Python SDK
FastAPI
Redis Queue / RQ, 可选
Postgres, 可选
React / Vite / Tailwind, 可选
Docker
GitHub Actions

4. 项目目录结构

4.1 P0 目录结构

```
insightflow-agent/
├── README.md
├── REQUIREMENTS.md
├── DEVELOPMENT_PLAN.md
├── USE_CASES.md
├── OUTPUTS_AND_ARTIFACTS.md
├── REFERENCE_PROJECTS.md
├── requirements.txt
├── .env.example
├── app.py
├──
├── data/
│   ├── ecommerce.db
│   ├── seed_data.py
│   └── metrics.yaml
├──
├── agents/
│   ├── supervisor.py
│   ├── schema_agent.py
│   ├── metric_agent.py
│   ├── sql_generator.py
│   ├── sql_reviewer.py
│   ├── error_fixer.py
│   └── insight_agent.py
├──
├── tools/
│   ├── schema_tool.py
│   ├── metric_tool.py
│   ├── sql_validator.py
│   ├── sql_executor.py
│   └── trace_logger.py
├──
├── graph/
│   ├── state.py
│   ├── nodes.py
│   └── workflow.py
├──
├── eval/
│   ├── test_questions.json
│   ├── run_eval.py
│   └── report.md
├──
├── logs/
│   └── traces/
├──
└── reports/
```

```
|   |—— charts/
|   |—— markdown/
|
|—— tests/
|   |—— test_schema_tool.py
|   |—— test_metric_tool.py
|   |—— test_sql_validator.py
|   |—— test_sql_executor.py
|   |—— test_workflow.py
|   |—— test_eval.py
```

4.2 P1 扩展目录

```
agents/
|—— context_retriever.py
|—— evidence_validator.py
|—— chart_agent.py
|—— report_agent.py

tools/
|—— context_tool.py
|—— python_analysis_tool.py
|—— chart_tool.py
|—— report_tool.py

data/
|—— business_rules.md
|—— table_docs.md
|—— sql_examples.json
```

4.3 P2 扩展目录

```
agents/
|—— report_supervisor.py
|—— action_planner.py
|—— risk_assessor.py
|—— action_verifier.py

tools/
|—— action_tool.py
|—— approval_tool.py
|—— audit_logger.py

SQLite tables:
|—— reports
```

```
|— tasks
|— metric_alerts
|— approvals
|— audit_logs
|— scheduled_runs
```

4.4 P3 扩展目录

```
mcp_servers/
|— database_server.py
|— report_server.py
|— action_server.py

api/
|— main.py

dashboard/
Dockerfile
docker-compose.yml
.github/workflows/
```

5. P0 开发计划：Agentic SQL Core

P0 目标：

先证明 InsightFlow 不是普通 Text2SQL，而是 Multi-Agent + Tool Calling + SQL Execution Feedback Workflow。

P0 必须完成：

```
SQLite 电商数据库
Schema Tool
Metric Tool
SQL Validator
SQL Executor
Trace Logger
SQL Generator Agent
SQL Reviewer Agent
Error Fix Agent
Insight Agent
LangGraph workflow
Streamlit Demo
20 个 Eval Case
```

```
pytest
README
```

5.1 Task 0: 项目初始化

目标

创建基础项目结构，保证项目可以安装依赖、运行测试、启动 Streamlit。

文件

```
requirements.txt
.env.example
README.md
app.py
agents/
tools/
graph/
eval/
tests/
data/
logs/
reports/
```

验收标准

```
pip install -r requirements.txt 成功
pytest 可以运行
streamlit run app.py 可以启动空页面
目录结构完整
```

5.2 Task 1: 构建电商 SQLite 数据库

目标

创建用于 P0 的电商示例数据库。

文件

```
data/seed_data.py
data/ecommerce.db
```


表

```
users
orders
order_items
products
categories
```

数据要求

```
至少 100 个用户
至少 500 个订单
至少 1000 条 order_items
至少 30 个商品
至少 5 个品类
订单覆盖最近 6-12 个月
orders.status 包含 paid / cancelled / refunded
```

验收标准

```
python data/seed_data.py 可以生成 ecommerce.db
数据库中的所有表都有数据
可以手动执行基础 SQL 查询
```

5.3 Task 2: 实现 Metric Definition

目标

用 `metrics.yaml` 定义业务指标口径。

文件

```
data/metrics.yaml
tools/metric_tool.py
```

指标

```
gmv
order_count
aov
category_gmv
product_sales
```

示例

```
gmV:  
  name: "销售额"  
  formula: "SUM(order_items.quantity * order_items.unit_price)"  
  required_filters:  
    - "orders.status = 'paid'"
```

工具接口

```
retrieve_metric_definition(question: str) -> dict
```

验收标准

用户问题包含“销售额”时返回 GMV 定义
返回 formula 和 required_filters
metric tool 调用写入 trace

5.4 Task 3: 实现 Schema Tool

目标

实现数据库 Schema 读取工具。

文件

```
tools/schema_tool.py
```

工具接口

```
get_database_schema(db_path: str) -> dict
```

输出

```
{  
  "success": true,  
  "tables": [  
    {  
      "table_name": "orders",  
      "columns": [  
        {"name": "id", "type": "INTEGER"},  
        {"name": "order_date", "type": "TEXT"}  
      ]  
    }  
  ]  
}
```

```
}  
]  
}
```

验收标准

能读取所有表
能读取字段名和类型
能格式化成 prompt-friendly schema_text
测试覆盖正常数据库和空数据库

5.5 Task 4: 实现 SQL Validator

目标

实现执行前 SQL 安全与业务校验。

文件

```
tools/sql_validator.py
```

参考项目

局部参考 [azain47/Multi-Agent-Text2SQL-System](#) 的 parser-based SQL validation 思路。

检查项

只允许 SELECT
禁止 DROP / DELETE / UPDATE / INSERT / ALTER / TRUNCATE
禁止多语句
表名存在
字段名存在
必须有 LIMIT
敏感字段拦截
GMV 口径检查
orders.status = 'paid' 检查

工具接口

```
validate_sql(sql: str, schema: dict, metric_context: dict | None = None) -> dict
```

输出

```
{
  "approved": true,
  "risk_level": "low",
  "issues": [],
  "checks": {
    "select_only": true,
    "tables_exist": true,
    "columns_exist": true,
    "has_limit": true,
    "metric_formula_correct": true
  }
}
```

验收标准

危险 SQL 必须 rejected
字段不存在必须 detected
缺少 LIMIT 必须 warning 或自动补充
指标口径错误必须 detected
validate_sql failed 时 workflow 不能进入 run_sql

5.6 Task 5: 实现 SQL Executor

目标

实现真实 SQL 执行工具。

文件

```
tools/sql_executor.py
```

参考项目

重点参考 [adamfaik/sql-agent](#) 的 SQLite 执行、timeout、row limit、error capture 思路。

工具接口

```
run_sql(db_path: str, sql: str, timeout_seconds: int = 5, max_rows: int = 100) -> dict
```

成功输出

```
{
  "success": true,
  "columns": ["product_name", "sales"],
  "rows": [
    ["Camera A", 128000]
  ],
  "row_count": 1,
  "execution_time_ms": 34
}
```

失败输出

```
{
  "success": false,
  "error": "no such column: oi.price",
  "execution_time_ms": 11
}
```

验收标准

只允许 SELECT
真实连接 SQLite 执行
最多返回 100 行
执行失败返回 error，不抛出未捕获异常
执行结果写入 State 和 Trace

5.7 Task 6: 实现 Trace Logger

目标

记录每次 Agent run 的节点和工具调用。

文件

```
tools/trace_logger.py
logs/traces/
```

接口

```
append_trace(state: AgentState, event: dict) -> AgentState
save_trace(run_id: str, trace: list[dict]) -> str
```

Trace 字段

```
run_id
session_id
node
tool_name
tool_input_summary
tool_output_summary
status
latency_ms
error_type
retry_count
timestamp
```

验收标准

每次 run 生成 logs/traces/{run_id}.json
每个工具调用都有记录
失败和 retry 都能在 trace 中看到

5.8 Task 7: 实现 P0 Agents

文件

```
agents/supervisor.py
agents/schema_agent.py
agents/metric_agent.py
agents/sql_generator.py
agents/sql_reviewer.py
agents/error_fixer.py
agents/insight_agent.py
```

Agent 职责

Supervisor Agent: 判断任务类型, 初始化 run
Schema Agent: 调用 get_database_schema()
Metric Agent: 调用 retrieve_metric_definition()
SQL Generator Agent: 生成 SQL
SQL Reviewer Agent: 调用 validate_sql()
Error Fix Agent: 根据 error + schema 修复 SQL
Insight Agent: 基于 execution_result 生成回答

结构化输出要求

SQL Generator 输出:

```
{
  "sql": "...",
  "tables": [],
  "metrics": [],
  "reason": "..."
}
```

Error Fix Agent 输出:

```
{
  "fixed_sql": "...",
  "fix_reason": "...",
  "retry_count": 1
}
```

Insight Agent 输出:

```
{
  "final_answer": "...",
  "data_used": true
}
```

验收标准

Agent 输出可 parse
Agent 不直接执行数据库
Agent 通过 Tool 获取外部结果
最终回答必须基于 execution_result

5.9 Task 8: 实现 LangGraph Workflow

文件

```
graph/state.py
graph/nodes.py
graph/workflow.py
```

P0 State

```
class AgentState(TypedDict):
    run_id: str
    session_id: str
    user_question: str
```

```
database_schema: dict
schema_text: str

metric_context: dict
selected_tables: list[str]

generated_sql: str
sql_reason: str
review_result: dict

execution_result: dict
error_message: str
fixed_sql: str
retry_count: int

final_answer: str
trace: list[dict]
status: str
```

条件边

```
SQL Reviewer:
approved = true -> SQL Executor
approved = false -> SQL Generator

SQL Executor:
success = true -> Insight Agent
success = false and retry_count < 1 -> Error Fix Agent
success = false and retry_count >= 1 -> Fail Response
```

验收标准

```
workflow 可以完整执行
SQL 执行失败后能进入 Error Fix Agent
最多 retry 1 次
失败后返回解释，不编造答案
```

5.10 Task 9: 实现 Streamlit Demo

文件

```
app.py
```


页面模块

用户问题输入区
执行状态区
Agent Steps
Generated SQL
SQL Review Result
SQL Execution Result
Error Repair Process
Final Answer
Trace Viewer
Eval Entry

参考项目

参考 `adamfaik/sql-agent` 的 Streamlit glass-box demo 思路。

验收标准

可以输入中文问题
可以看到 SQL
可以看到 review_result
可以看到 execution_result
可以看到错误修复过程
可以打开 trace.json

5.11 Task 10: 实现 P0 Eval

文件

eval/test_questions.json
eval/run_eval.py
eval/report.md

测试集

20 个 case，覆盖：

基础 SQL 查询
销售额排名
品类统计
时间趋势
销售下降分析
SQL 错误修复

危险 SQL 拦截
指标口径校验

指标

SQL execution success rate
SQL first-pass success rate
SQL repair success rate
dangerous SQL block rate
metric definition accuracy
average tool calls
average latency
failure type distribution

验收标准

python eval/run_eval.py 可以执行
生成 eval/report.md
报告包含成功率、修复率、危险 SQL 拦截率、失败类型

6. P1 开发计划：Reliable Analysis & Report Core

P1 目标：

系统不只是会查 SQL，还能生成可追溯的业务分析产物。

6.1 Task 11：实现 Business Context Retrieval

文件

data/business_rules.md
data/table_docs.md
data/sql_examples.json
tools/context_tool.py
agents/context_retriever.py

工具

retrieve_business_context(question: str) -> dict

验收标准

能返回相关业务规则
能返回历史 SQL 示例
能返回字段说明
结果写入 State

6.2 Task 12: 实现 Evidence Validator

文件

agents/evidence_validator.py
tools/evidence_tool.py, 可选

功能

区分:

data-supported findings
hypotheses
unsupported claims

输出

```
{  
  "data_supported_findings": [],  
  "hypotheses": [],  
  "unsupported_claims_blocked": []  
}
```

验收标准

没有数据支持的原因不能写成确定结论
报告中必须区分 findings 和 hypotheses
Eval 可以统计 unsupported_claim_rate

6.3 Task 13: 实现 Chart Agent

文件

```
agents/chart_agent.py  
tools/chart_tool.py
```

参考项目

参考 [mallahyari/langgraph-sql-agent](#) 的 Visualization Planner 思路。

工具

```
generate_chart(data: dict, chart_spec: dict) -> dict
```

支持

```
bar  
line  
pie, 可选
```

验收标准

```
排名问题生成 bar  
趋势问题生成 line  
生成图表文件到 reports/charts/  
chart_path 写入 State
```

6.4 Task 14: 实现 Report Agent

文件

```
agents/report_agent.py  
tools/report_tool.py
```

工具

```
save_report(run_id: str, report_content: str) -> dict
```

报告结构

用户问题
使用指标
执行 SQL
查询结果摘要
数据支持结论
需要进一步验证的假设
图表路径
下一步建议
Trace 路径

验收标准

生成 reports/markdown/{run_id}_report.md
报告结论可追溯到 SQL 和 execution_result
报告包含 trace 路径

7. P2 开发计划：Business Review & Action Workflow

P2 目标：

支持经营周报、复盘报告和轻量行动型工具调用。

7.1 Task 15：实现 Business Review Report

文件

agents/report_supervisor.py

输入示例

帮我生成一份本周电商经营分析周报，包括销售额、订单量、Top 商品、下降品类、渠道表现和运营建议。

Report Supervisor 拆解

本周 GMV
本周订单量
本周客单价
Top 商品
Top 品类
下降最多品类
渠道表现
异常点
下周建议

验收标准

系统可以为一个周报任务执行多个 SQL 子任务
每个子任务都有 SQL、review、execution result
最终保存 weekly_business_report.md

7.2 Task 16: 实现 Action Workflow

文件

agents/action_planner.py
agents/risk_assessor.py
agents/action_verifier.py
tools/action_tool.py
tools/approval_tool.py
tools/audit_logger.py

工具

create_task()
create_metric_alert()
create_email_draft()
verify_action_execution()
log_audit_event()

验收标准

Action Planner 可以生成结构化 action_plan
Risk Assessor 可以判断是否需要审批
Approval Gate 可以阻止未审批动作
create_task / create_metric_alert 写入 SQLite

Action Verifier 可以确认任务和告警创建成功
Audit Log 保存审批和执行过程

8. P3 开发计划：MCP & Engineering Core

P3 目标：

增强工具标准化和工程化能力。

8.1 Task 17: MCP Tool Layer

MCP Server

```
database-mcp-server:  
  get_database_schema  
  get_sample_rows  
  run_sql
```

```
report-mcp-server:  
  generate_chart  
  save_report
```

```
action-mcp-server:  
  create_task  
  create_metric_alert  
  create_email_draft
```

暂不 MCP 化

```
validate_sql  
permission_checker  
log_trace  
eval_runner
```

原因：

它们更适合作为系统内部安全、审计和评测模块。

8.2 Task 18: FastAPI + Async Run API

API

```
POST /api/runs
GET /api/runs/{run_id}
GET /api/runs/{run_id}/trace
GET /api/runs/{run_id}/events
POST /api/runs/{run_id}/cancel
```

状态

```
queued
running
waiting_for_approval
completed
failed
cancelled
```

8.3 Task 19: Trace Dashboard

展示

```
Agent 节点耗时
工具调用次数
SQL 执行耗时
SQL 修复次数
失败类型分布
Eval 成功率
Action 审批记录
Audit Log
```

9. 开发顺序总结

第一阶段：P0

1. 项目初始化
2. seed SQLite 数据库
3. metrics.yaml
4. Schema Tool
5. Metric Tool
6. SQL Validator

7. SQL Executor
8. Trace Logger
9. PO Agents
10. LangGraph Workflow
11. Streamlit Demo
12. Eval
13. README

第二阶段：P1

1. Business Context Retrieval
2. Evidence Validator
3. Chart Agent
4. Report Agent
5. Report Eval

第三阶段：P2

1. Business Review Report
2. Action Planner
3. Risk Assessor
4. Approval Gate
5. Task / Alert Tools
6. Action Verifier
7. Audit Log

第四阶段：P3

1. MCP Tool Layer
2. FastAPI Run API
3. Async Jobs
4. Trace Dashboard
5. Docker / CI

10. P0 完成标准

P0 完成后，项目必须满足：

用户可以通过 Streamlit 输入中文业务问题。
系统可以调用 `get_database_schema` 获取真实 Schema。
系统可以调用 `retrieve_metric_definition` 获取 GMV 等指标定义。
SQL Generator 可以生成 SELECT SQL。
SQL Reviewer 可以调用 `validate_sql` 校验 SQL。

危险 SQL 会被拒绝，不进入 run_sql。
SQL Executor 可以调用 run_sql 真实执行 SQL。
SQL 执行失败后，Error Fix Agent 可以修复一次。
修复后的 SQL 会重新经过 validate_sql 和 run_sql。
最终回答基于 execution_result，不编造。
每次 run 都有完整 trace.json。
eval/run_eval.py 可以运行 20 个测试问题。
README 有启动说明、架构说明、Demo 示例和 Eval 结果。

11. 最终开发原则

不要一开始做大而全。
不要先做周报、MCP、异步和 ActionOps。
先把 P0 的 Multi-Agent + Tool Calling + SQL Execution Feedback 做扎实。
P1 再做报告。
P2 再做周报和行动。
P3 再做工程化。

项目第一原则：

不要堆功能，要突出 Multi-Agent 和 Tool Calling。